

技术方案

企业级 AI Agent 网关 · v3.0

1. new-api 改造全景

Code block	
1 new-api 现有代码	本次改造
2	
3 controller/	✓ 复用，少量扩展
4 middleware/auth.go	🔧 扩展：注入 tenant/user/seat
5 middleware/rate_limit.go	🔧 扩展：多级限流 (seat + global)
6 model/channel.go	🔧 新增 channel type 45
7 relay/channel/helper.go	🔧 注册 Copilot Adaptor
8 router/	🔧 新增 /v1/messages 路由
9	
10 新增包 relay/channel/copilot/:	
11 adaptor.go	渠道 Adaptor (核心路由)
12 token.go	Token Exchange + 主动刷新 + 两级缓存
13 headers.go	完整 IDE Headers (含 x-request-id 等)
14 vscode_version.go	动态 VSCode 版本 (AUR + 24h 缓存)
15 anthropic_to_openai.go	Anthropic 请求 → OpenAI 格式
16 openai_to_anthropic.go	OpenAI 响应 → Anthropic 格式 (非流式)
17 stream_translator.go	OpenAI SSE → Anthropic SSE 状态机 (流式)
18 model.go	动态模型列表、max_tokens 充、模型名规范化
19 types.go	所有结构体定义
20	
21 新增业务层:	
22 service/seat/binding.go	Seat 映射 + 加密
23 service/audit/producer.go	旁路审计异步写入
24 model/seat_binding.go	ORM
25 model/billing_event.go	ORM
26 migrations/	DB schema

2. 核心模块设计

2.1 Copilot Adaptor (adaptor.go)

实现 new-api ChannelAdaptor 接口，是整个集成的唯一入口。

Code block

```
1  请求进入 Adaptor:
2    1. 从 meta 取 GitHub Token (Seat 映射已在 middleware 完成)
3    2. GetCopilotToken(githubToken) → copilot_token + base_url (带缓存)
4    3. 根据 meta.Mode / 路径判断:
5        - chat/completions → 透传 (模型名规范化)
6        - messages (Anthropic) → TranslateAnthropicToOpenAI → 转发 → 响应转换
7        - embeddings → 透传
8    4. SetupRequestHeader: 注入完整 IDE Headers
9    5. DoResponse:
10       - 非 Claude / 非 messages 路径: 透传 resp.Body
11       - messages 路径 + stream: 流式状态机转换
12       - messages 路径 + 非 stream: TranslateOpenAIToAnthropic
```

2.2 Token Exchange + 主动刷新 (token.go)

两级缓存 + 主动刷新 (参考 copilot-api) :

Code block

```
1  请求来时:
2    检查进程内 sync.Map
3    命中且未过期 (expires_at - 5min > now) → 直接返回
4    未命中或过期:
5        尝试 Redis (key = copilot:token:sha256(githubToken))
6        命中 → 写回 sync.Map, 返回
7        未命中 → 发起 Token Exchange HTTP 请求
```

```

8      成功 → 写 sync.Map + Redis (TTL = expires_at - now - 5min)
9      启动后台 goroutine 按 refresh_in - 60s 主动刷新
10
11 主动刷新 goroutine:
12     sleep(refresh_in - 60s)
13     重新 Token Exchange
14     更新 sync.Map + Redis
15     继续下一次 refresh 调度
16
17 错误处理:
18     401 → 返回用户 401 (GitHub Token 无效)
19     403 → 返回用户 403 (无 Copilot 订阅)
20     429 → 指数退避重试 3 次
21     5xx → 有缓存则降级使用旧 token, 无则 502

```

2.3 完整 IDE Headers (headers.go)

Code block

```

1  必须注入的 10 个 Header (任意缺少可能 403 或行为异常) :
2
3  Authorization          Bearer {copilot_token}
4  Content-Type           application/json
5  editor-version         vscode/{动态版本}      ← 关键, 需定期更新
6  editor-plugin-version  copilot-chat/0.26.7
7  user-agent             GitHubCopilotChat/0.26.7
8  copilot-integration-id vscode-chat
9  openai-intent          conversation-panel
10 x-github-api-version    2025-04-01
11 x-request-id           {uuid.NewString()}      ← 每次唯一
12 x-vscode-user-agent-library-version electron-fetch
13
14 条件 Header:
15 X-Initiator            user / agent (按消息历史判断)
16 copilot-vision-request true (含图片时)

```

版本号热更新机制:

```
1 优先级：
2 1. 环境变量 COPILOT_EDITOR_VERSION (立即生效，无需重启)
3 2. 动态从 AUR 拉取 (24h 缓存)
4 3. 硬编码 fallback ("1.104.3")
5
6 监控告警：
7 403 错误率 > 1% → 立即告警
8 → 检查 cc-switch CHANGELOG → 更新环境变量 → 无需重启
```

2.4 Anthropic ↔ OpenAI 协议转换

请求转换要点 (anthropic_to_openai.go)

Code block

```
1  难点一：消息顺序重排
2    Anthropic user 消息可能包含 tool_result block
3    tool_result 必须转换为 role="tool" 且排在用户内容之前
4    否则 OpenAI API 报序列错误
5
6  难点二：thinking block 处理
7    Anthropic assistant 消息中的 thinking block 无 OpenAI 对应
8    必须合并进 content 文本，否则多轮对话上下文断裂
9
10  难点三：max_tokens 必填
11    Anthropic 协议中 max_tokens 必填
12    OpenAI 可选；但当外部是 Anthropic 客户端时，网关必须确保转发时有合理的 max_tokens
13    从 /models 动态获取该模型的 max_output_tokens, fallback 8192
14
15  难点四：模型名规范化
16    claude-sonnet-4-5 / claude-sonnet-4-20250514 → claude-sonnet-4
17    claude-opus-4-5 → claude-opus-4
18    Copilot 不接受带子版本号的模型名，否则 400
```

流式响应转换 (stream_translator.go)

```

1  状态机维护：
2      MessageStartSent  bool          是否已发出 message_start
3      ContentBlockIndex int          当前 block 编号（从 0 开始递增）
4      ContentBlockOpen  bool          当前是否有 block 开着
5      ToolCallBlocks    map[int]...   OpenAI tool_call index → Anthropic block 映射
6
7  转换规则：
8      首 chunk                → message_start (含 usage 初始值)
9      delta.content != ""     → 若有 tool block 开着则先 stop, 开 text block, 发
      text_delta
10     delta.tool_calls 新 ID   → 若有 block 开着则先 stop, 开 tool_use block
11     delta.tool_calls 参数    → input_json_delta 追加到对应 tool block
12     finish_reason != ""     → 关所有开着的 block, 发 message_delta + message_stop
13     [DONE]                  → 结束
14
15 关键边缘 case：
16     同一 chunk 中同时有 content 和 tool_calls → 先处理 content, 再处理 tool_calls
17     多个并发 tool_calls (parallel tool calls) → 通过 index 区分, 各自维护 block

```

2.5 Seat 映射 (service/seat/binding.go)

Code block

```

1  请求链路：
2      API Key → 解析 tenant_id (from new-api 现有 key 体系)
3      X-User-Id Header → user_id
4      查询 seat_bindings (Redis 10min 缓存 → PG 兜底)
5      AES-256-GCM 解密 → github_token
6      注入 meta.APIKey = github_token
7
8  加密方案：
9      算法：AES-256-GCM
10     密钥管理：AWS KMS / GCP KMS (密钥不落本地磁盘)
11     存储格式：base64(iv || ciphertext || tag)
12     解密时机：每次请求时实时调用 KMS 解密, 明文不缓存到 Redis

```

2.6 多级限流 (middleware/rate_limit.go 扩展)

```

1 Redis Key:
2   rl:seat:{seat_id}:conc      并发数 (INCR/DECR)
3   rl:seat:{seat_id}:qpm      每分钟请求数 (滑动窗口)
4   rl:global:conc             全局并发数
5
6 Lua 原子脚本 (一次 roundtrip 完成三个检查) :
7   1. seat 并发 >= seat_conc_limit → 返回 429 (SEAT_CONCURRENT)
8   2. seat QPM >= seat_qpm_limit   → 返回 429 (SEAT_QPM)
9   3. global 并发 >= global_limit  → 返回 503 (GLOBAL_OVERLOAD)
10  全部通过 → INCR seat conc + global conc
11
12 defer (请求结束无论成败) :
13   DECR rl:seat:{seat_id}:conc
14   DECR rl:global:conc
15
16 Fail-open:
17   Redis 不可用 → sync.Map 本地计数 (上限 = Redis 配置 × 50%)
18   记录 WARN 级审计日志, 供事后分析

```

2.7 旁路审计 (service/audit/producer.go)

Code block

```

1  type AuditEvent struct {
2      TenantID    string
3      UserID      string
4      SeatID      string
5      Model       string
6      Protocol    string    // "openai" / "anthropic"
7      InputTokens int
8      OutputTokens int
9      StatusCode  int
10     DurationMS   int64
11     ClientIPHash string    // sha256, 不存原始 IP
12     CreatedAt    time.Time
13 }
14
15 // 主链路只做: ch <- event (非阻塞, channel 满时采样丢弃)
16 // 后台 goroutine 消费: 批量写 PG (每 100 条或 5 秒 flush 一次)
17 // PG 30 天后归档对象存储 (定时任务)

```

3. 数据库 Schema

Code block

```
1  -- Seat 绑定 (核心)
2  CREATE TABLE seat_bindings (
3      id                BIGSERIAL PRIMARY KEY,
4      tenant_id         UUID NOT NULL,
5      user_id           VARCHAR(128) NOT NULL,
6      seat_id           VARCHAR(128) NOT NULL UNIQUE,
7      encrypted_token   TEXT NOT NULL,
8      account_type      VARCHAR(32) DEFAULT 'individual',
9      status            VARCHAR(32) DEFAULT 'active',
10     bound_at          TIMESTAMPTZ DEFAULT NOW(),
11     last_used_at      TIMESTAMPTZ,
12     UNIQUE (tenant_id, user_id)
13 );
14 CREATE INDEX idx_seat_lookup ON seat_bindings(tenant_id, user_id);
15
16 -- 计费事件
17 CREATE TABLE billing_events (
18     id                BIGSERIAL PRIMARY KEY,
19     tenant_id         UUID NOT NULL,
20     user_id           VARCHAR(128) NOT NULL,
21     seat_id           VARCHAR(128) NOT NULL,
22     request_id        VARCHAR(128) NOT NULL UNIQUE,
23     model             VARCHAR(64),
24     protocol          VARCHAR(16),
25     status_code       INTEGER,
26     input_tokens       INTEGER DEFAULT 0,
27     output_tokens      INTEGER DEFAULT 0,
28     duration_ms       INTEGER DEFAULT 0,
29     billed_at         TIMESTAMPTZ DEFAULT NOW(),
30     billing_cycle     VARCHAR(7)
31 );
32 CREATE INDEX idx_billing_tenant_cycle ON billing_events(tenant_id,
33     billing_cycle);
34
35 -- 审计事件 (热数据, 30 天后归档)
36 CREATE TABLE audit_events (
37     id                BIGSERIAL PRIMARY KEY,
```

```

38     user_id          VARCHAR(128),
39     seat_id          VARCHAR(128),
40     model             VARCHAR(64),
41     protocol          VARCHAR(16),
42     input_tokens      INTEGER,
43     output_tokens     INTEGER,
44     status_code       INTEGER,
45     duration_ms       INTEGER,
46     client_ip_hash    VARCHAR(64),
47     created_at        TIMESTAMPTZ DEFAULT NOW()
48 );
49 CREATE INDEX idx_audit_tenant_time ON audit_events(tenant_id, created_at DESC);
50
51 -- Copilot Token 缓存 (Redis 优先, PG 作 Redis 不可用时的兜底)
52 CREATE TABLE copilot_token_cache (
53     github_token_hash VARCHAR(64) PRIMARY KEY,
54     copilot_token      TEXT NOT NULL,
55     base_url           VARCHAR(256),
56     expires_at         TIMESTAMPTZ NOT NULL,
57     refresh_in         INTEGER,
58     updated_at         TIMESTAMPTZ DEFAULT NOW()
59 );

```

4. 关键配置项

Code block

```

1  # 环境变量 (Kubernetes Secret / ConfigMap)
2
3  # IDE Headers 版本 (热更新, 无需重启)
4  COPILOT_EDITOR_VERSION=vscode/1.104.3      # 可随时更新
5  COPILOT_PLUGIN_VERSION=copilot-chat/0.26.7
6  COPILOT_USER_AGENT=GitHubCopilotChat/0.26.7
7
8  # KMS (GitHub Token 解密)
9  KMS_KEY_ID=arn:aws:kms:us-east-1:xxx:key/xxx
10 KMS_REGION=us-east-1
11
12 # Redis

```



```
13 REDIS_URL=redis://...
14
15 # 限流（可按租户覆盖）
16 RATE_LIMIT_SEAT_CONCURRENT=3
17 RATE_LIMIT_SEAT_QPM=60
18 RATE_LIMIT_GLOBAL_CONCURRENT=100
19
20 # 模型列表缓存 TTL（分钟）
21 COPILOT_MODELS_CACHE_TTL=30
22
23 # VSCode 版本缓存 TTL（小时）
24 VSCODE_VERSION_CACHE_TTL=24
```

5. 监控告警

指标	告警阈值	含义	响应动作
Token Exchange 403 率	> 1%	IDE Headers 版本过时	更新 COPILOT_EDITOR_VERSION 环境变量
Token Exchange 429 率	> 5%	Token 换取过于频繁	检查 refresh_in 刷新逻辑
Token Exchange P95 耗时	> 500ms	GitHub API 响应慢	检查网络，扩展 Redis 缓存
Seat 限流命中率	> 20%	并发配置偏低	调整 RATE_LIMIT_SEAT_CONCURRENT
全局并发利用率	> 80%	需扩容	增加 Deployment 副本
Redis 不可用时长	> 30s	Fail-open 已启用	紧急恢复 Redis
Anthropic 流式转换错误率	> 0.1%	状态机边缘 case	查日志，修 stream_translator

6. 工程目录结构

Code block

1	new-api (fork)	
2	├─ relay/channel/copilot/	 全部新增 (核心)
3	│ └─ adaptor.go	渠道 Adaptor
4	│ └─ token.go	Token Exchange + 主动刷新
5	│ └─ headers.go	完整 IDE Headers
6	│ └─ vscode_version.go	动态 VSCode 版本 (AUR)
7	│ └─ anthropic_to_openai.go	请求转换 (含 tool/thinking/image)
8	│ └─ openai_to_anthropic.go	非流式响应转换
9	│ └─ stream_translator.go	流式状态机转换
10	│ └─ model.go	动态模型列表 + 名称规范化
11	│ └─ types.go	所有结构体
12	├─ service/	
13	│ └─ seat/binding.go	 Seat 映射 + 解密
14	│ └─ audit/producer.go	 旁路审计
15	├─ model/	
16	│ └─ seat_binding.go	 ORM
17	│ └─ billing_event.go	 ORM
18	├─ middleware/	
19	│ └─ auth.go	 扩展 tenant/user/seat 注入
20	│ └─ rate_limit.go	 扩展多级限流
21	├─ router/	 新增 /v1/messages 路由
22	├─ migrations/	 schema migration 文件
23	├─ deploy/k8s/	 Kubernetes manifests
24	└─ docs/copilot-channel.md	 使用说明